



ASP.NET CORE ESSENTIALS

Module 6-10

Table of Contents

Module 6: Database-First EF Core in ASP.NET Core	4
6.1 What is Database-First Approach?	4
6.2 Setting Up EF Core for Database-First	4
Step 1: Install EF Core Packages	4
Step 2: Create a Connection String	5
6.3 Scaffolding the Database	5
Parameters Explained	5
6.4 Understanding Generated Files	5
6.5 Configuring DbContext in ASP.NET Core	6
Step 1 — Register it	7
Step 2 — Ask for it in your controller or service	7
6.6 Performing CRUD Operations	7
6.6.1 Read Data	7
6.6.2 Create Data	8
6.6.3 Update Data	8
6.6.4 Delete Data	8
6.7 Partial Classes and Re-Scaffolding	8
6.8 Best Practices for Database-First EF Core	9
6.9 Summary of Module 6	9
Module 7: Code-First EF Core in ASP.NET Core	10
7.1 What Is Code-First Approach?	10
7.2 Setting Up EF Core for Code-First	10
Step 1: Install EF Core Packages	10
Step 2: Define Model Classes	10
Step 3: Create DbContext	11
Step 4: Configure DbContext in ASP.NET Core	11
7.3 Creating the Database Using Migrations	12
Step 1: Add Initial Migration	12
Step 2: Apply Migration	12
7.4 Updating Models and Migrations	12
7.5 Relationships in Code-First EF	12
7.5.1 One-to-Many Example	12
7.5.2 Many-to-Many Example	13

7.6 Data Seeding in Code-First EF	14
7.7 Performing CRUD Operations	14
Create	14
Read	14
Update	14
Delete	14
7.8 Advantages of Code-First EF Core	15
7.9 Best Practices	15
7.10 Summary of Code-First EF Core	15
Module 8: Session & Cookie Management in ASP.NET Core	16
8.1 Understanding Session vs Cookies	16
8.2 Configuring Session in ASP.NET Core	17
Step 1: Install Package (if needed)	17
Step 2: Configure in Program.cs	17
8.3 Using Session in Controllers	17
8.3.1 Storing Data	17
8.3.2 Reading Data	17
8.3.3 Removing Data	17
Example: Login Session	18
8.4 Creating and Managing Cookies	18
8.4.1 Setting Cookies	18
8.4.2 Reading Cookies	18
8.4.3 Deleting Cookies	18
8.5 Persistent vs Temporary Cookies	18
8.6 Session vs Cookie Example	19
8.7 Security Considerations	19
8.8 Best Practices	19
8.9 Summary of Session & Cookie Management	19
Module 9: DTOs and AutoMapper in ASP.NET Core	21
9.1 What Are DTOs (Data Transfer Objects)?	21
Benefits of Using DTOs	21
9.2 Example: Entity vs DTO	21
9.3 Creating DTOs in ASP.NET Core	22
9.3.1 Create DTOs for CRUD Operations	22

9.3.2 Why Separate DTOs?	22
9.4 Introduction to AutoMapper	23
9.5 Installing AutoMapper	23
9.6 Configuring AutoMapper in ASP.NET Core	23
Step 1: Create a Mapping Profile.....	23
Step 2: Register AutoMapper.....	23
9.7 Using AutoMapper in Controllers	24
9.7.1 Inject IMapper.....	24
9.7.2 Mapping Entities to DTOs	24
9.7.3 Mapping DTO to Entity	24
9.8 Advanced AutoMapper Features	24
Example: Custom Mapping.....	25
9.9 Best Practices for DTOs & AutoMapper	25
9.10 Summary of DTOs and AutoMapper	25
Module 10: Session & Cookie-Based Authentication with Custom Authorization in ASP.NET Core MVC	26
10.1 Understanding Session and Cookie-Based Authentication	26
10.2 Setting Up Session and Cookie Authentication	27
Step 1: Configure Services in Program.cs	27
Step 2: Enable Middleware	27
10.3 User Login: Session + Cookie	27
Step 1: Login Action	27
Step 2: Logout Action	28
10.4 Custom Authorization Attribute.....	28
Step 1: Create the Filter Attribute.....	28
Step 2: Apply the Attribute	29
10.5 Access Denied Page	29
10.6 Advantages of This Approach.....	30
10.7 Best Practices	30
10.8 Summary of Module 10	30

Module 6: Database-First EF Core in ASP.NET Core

Entity Framework Core (EF Core) is a modern Object-Relational Mapper (ORM) from Microsoft that allows developers to work with databases using **C# classes instead of raw SQL queries**. It simplifies database access, CRUD operations, and ensures type safety.

There are two primary approaches to using EF Core:

1. **Code-First** – You define the model classes first, then EF Core creates the database.
2. **Database-First** – You start with an existing database, and EF Core generates model classes and DbContext automatically.

In this chapter, we focus on **Database-First** approach.

6.1 What is Database-First Approach?

Database-First is ideal when:

- You have an **existing database** with tables, relationships, and constraints.
- You want to **generate entity classes and DbContext** automatically.
- You want to **avoid manually creating models** for each table.

EF Core uses the **Scaffold-DbContext** command to reverse-engineer the database.

6.2 Setting Up EF Core for Database-First

Step 1: Install EF Core Packages

For SQL Server, install these NuGet packages:

```
Microsoft.EntityFrameworkCore.SqlServer  
Microsoft.EntityFrameworkCore.Design  
Microsoft.EntityFrameworkCore
```

- `SqlServer` → EF Core provider for SQL Server

- Design → Required for scaffolding and migrations

Use Visual Studio NuGet Package Manager or CLI commands.

Install dotnet-ef command globally for future usage:

```
dotnet tool install --global dotnet-ef
```

Step 2: Create a Connection String

Add connection string in `appsettings.json`:

```
"ConnectionStrings": {  
  "SchoolDbContext": "Data Source=DESKTOP-GAKQQ5S\\SQLEXPRESS;Initial  
Catalog=SchoolDb;TrustServerCertificate=True;Integrated Security=True;"  
}
```

6.3 Scaffolding the Database

Use the **dotnet ef dbcontext scaffold** command to generate entity classes and DbContext:

```
dotnet ef dbcontext scaffold "Name=SchoolDbContext" --context-dir EF --output-dir EF/Tables  
Microsoft.EntityFrameworkCore.SqlServer
```

Parameters Explained

Parameter	Description
Connection string	Database to reverse-engineer
Provider	EF Core database provider (SQL Server, MySQL, etc.)
--output-dir	Folder to generate model classes
--context-dir	Folder to generate context class

6.4 Understanding Generated Files

After scaffolding, EF Core generates:

1. **DbContext Class** – Handles database connections and operations.
Example: `SchoolDbContext.cs`

```
public class SchoolDbContext : DbContext
```

```

{
    public SchoolDbContext(DbContextOptions<SchoolDbContext> options)
        : base(options)
    {
    }

    public DbSet<Student> Students { get; set; }
    public DbSet<Course> Courses { get; set; }
}

```

2. **Entity Classes** – Represent database tables.

Example: Student.cs

```

public partial class Student
{
    public int StudentId { get; set; }
    public string Name { get; set; }
    public DateTime DateOfBirth { get; set; }
}

```

These classes are ready to use for CRUD operations.

6.5 Configuring DbContext in ASP.NET Core

In Program.cs (ASP.NET Core 6+):

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<SchoolDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("SchoolDbContext ")));

builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseStaticFiles();
app.UseRouting();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}"
);

app.Run();

```

This enables **Dependency Injection** for DbContext across controllers and services.

You **do NOT manually create** the DbContext.
ASP.NET Core creates it automatically using **Dependency Injection (DI)**.

Step 1 — Register it

Program.cs

```
builder.Services.AddDbContext<SchoolDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("SchoolDbContext ")));
```

Step 2 — Ask for it in your controller or service

```
public class UserController : ControllerBase
{
    private readonly SchoolDbContext _context;

    public UserController(SchoolDbContext context)
    {
        _context = context;
    }

    public IActionResult Index()
    {
        var students = _context.Students.ToList();
        return View(students);
    }
}
```

ASP.NET Core automatically creates the object for you.
You never call `new SchoolDbContext()` manually.

6.6 Performing CRUD Operations

6.6.1 Read Data

```
public class StudentsController : Controller
{
    private readonly SchoolDbContext _context;

    public StudentsController(SchoolDbContext context)
    {
        _context = context;
    }

    public IActionResult Index()
```



```

    {
        var students = _context.Students.ToList();
        return View(students);
    }
}

```

6.6.2 Create Data

```

[HttpPost]
public IActionResult Create(Student student)
{
    if (ModelState.IsValid)
    {
        _context.Students.Add(student);
        _context.SaveChanges();
        return RedirectToAction("Index");
    }
    return View(student);
}

```

6.6.3 Update Data

```

[HttpPost]
public IActionResult Edit(Student student)
{
    if (ModelState.IsValid)
    {
        _context.Students.Update(student);
        _context.SaveChanges();
        return RedirectToAction("Index");
    }
    return View(student);
}

```

6.6.4 Delete Data

```

public IActionResult Delete(int id)
{
    var student = _context.Students.Find(id);
    if (student != null)
    {
        _context.Students.Remove(student);
        _context.SaveChanges();
    }
    return RedirectToAction("Index");
}

```

6.7 Partial Classes and Re-Scaffolding

- EF Core scaffolds **partial classes**
- You can **extend entity classes** without modifying generated code
- If the database changes, **re-run dotnet ef dbcontext scaffold** to update models
- Use --force parameter to overwrite existing files

```
dotnet ef dbcontext scaffold "Name=SchoolDbContext" --context-dir EF --output-dir EF/Tables  
Microsoft.EntityFrameworkCore.SqlServer --force
```

6.8 Best Practices for Database-First EF Core

1. **Keep generated models separate** (e.g., EF/Tables)
2. **Do not modify scaffolded classes directly** – extend using partial classes
3. **Use DTOs or ViewModels** when passing data to views or APIs
4. **Use Dependency Injection** for DbContext
5. **Use migrations cautiously** if database-first; primary schema should be maintained in database

6.9 Summary of Module 6

In this module, you learned:

- The **database-first approach** and when to use it
- How to scaffold an existing database using EF Core
- How to configure DbContext in ASP.NET Core
- Performing CRUD operations using scaffolded entities
- Best practices for maintaining scaffolded code and extending functionality

Database-First EF Core allows you to **leverage existing databases** and focus on **business logic**, without manually creating models or DbContext.

Module 7: Code-First EF Core in ASP.NET Core

While **Database-First** starts with an existing database, **Code-First** starts with **C# classes** and generates the database schema automatically. This approach is ideal when you are creating **new projects** or want **full control over the data model** from your application code.

Code-First EF Core enables **rapid development, migrations, and maintainable models**.

7.1 What Is Code-First Approach?

Code-First EF Core allows you to:

1. Define your **model classes** in C#
2. Define **DbContext** to represent your database
3. Use **migrations** to create or update the database automatically

Benefits:

- Full control over table design from code
 - Easy to maintain and version database changes
 - Can integrate seamlessly with CI/CD pipelines
-

7.2 Setting Up EF Core for Code-First

Step 1: Install EF Core Packages

For SQL Server:

```
Microsoft.EntityFrameworkCore.SqlServer  
Microsoft.EntityFrameworkCore.Design  
Microsoft.EntityFrameworkCore
```

Step 2: Define Model Classes

Example: Creating a **Student** model

```

using System.ComponentModel.DataAnnotations;

public class Student
{
    [Key]
    public int StudentId { get; set; }

    [Required]
    [StringLength(50)]
    public string Name { get; set; }

    [Range(10, 100)]
    public int Age { get; set; }

    public DateTime DateOfBirth { get; set; }
}

```

Step 3: Create DbContext

DbContext represents the database session:

```

using Microsoft.EntityFrameworkCore;

public class SchoolDbContext : DbContext
{
    public SchoolDbContext(DbContextOptions<SchoolDbContext> options)
        : base(options)
    {
    }

    public DbSet<Student> Students { get; set; }
}

```

Step 4: Configure DbContext in ASP.NET Core

In Program.cs:

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<SchoolDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("SchoolDbContext ")));

builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseStaticFiles();
app.UseRouting();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

```

```
app.Run();
```

7.3 Creating the Database Using Migrations

Step 1: Add Initial Migration

```
dotnet ef migrations add InitialCreate
```

- This generates a migration file with `Up()` and `Down()` methods
- `Up()` → defines schema changes to apply
- `Down()` → defines rollback logic

Step 2: Apply Migration

```
dotnet ef database update
```

- Creates database and tables based on `DbContext` and models
 - Uses connection string in `appsettings.json`
-

7.4 Updating Models and Migrations

When your model changes:

1. Modify the C# classes
2. Add a new migration:

```
dotnet ef migrations add AddEmailToStudent
```

3. Apply changes:

```
dotnet ef database update
```

EF Core will **alter tables automatically**.

7.5 Relationships in Code-First EF

7.5.1 One-to-Many Example

Models:

```
public class Course
{
    [Key]
    public int CourseId { get; set; }
    public string Title { get; set; }

    public List<Student> Students { get; set; } = new List<Student>();
}

public class Student
{
    [Key]
    public int StudentId { get; set; }
    public string Name { get; set; }

    public int CourseId { get; set; }
    public Course Course { get; set; }
}
```

EF Core will automatically create foreign key CourseId.

7.5.2 Many-to-Many Example

Models:

```
public class Student
{
    public int StudentId { get; set; }
    public string Name { get; set; }

    public List<StudentCourse> StudentCourses { get; set; }
}

public class Course
{
    public int CourseId { get; set; }
    public string Title { get; set; }

    public List<StudentCourse> StudentCourses { get; set; }
}

public class StudentCourse
{
    public int StudentId { get; set; }
    public Student Student { get; set; }

    public int CourseId { get; set; }
    public Course Course { get; set; }
}
```

```
}
```

EF Core automatically generates the **join table**.

7.6 Data Seeding in Code-First EF

You can insert default data during migration:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Course>().HasData(
        new Course { CourseId = 1, Title = "Mathematics" },
        new Course { CourseId = 2, Title = "Physics" }
    );
}
```

- Run migrations to populate default records
 - Useful for initial data in applications
-

7.7 Performing CRUD Operations

Create

```
var student = new Student { Name = "Alice", Age = 22 };
_context.Students.Add(student);
_context.SaveChanges();
```

Read

```
var students = _context.Students.ToList();
```

Update

```
var student = _context.Students.Find(1);
student.Name = "Alice Smith";
_context.SaveChanges();
```

Delete

```
var student = _context.Students.Find(1);
_context.Students.Remove(student);
_context.SaveChanges();
```

7.8 Advantages of Code-First EF Core

- **Full control** of models in code
 - **Automatic migrations** make database updates easy
 - **Supports complex relationships** and constraints
 - Works well with **DTOs, services, and APIs**
 - **Database evolves with the application**, not manually
-

7.9 Best Practices

1. **Keep models clean**; use data annotations or Fluent API
 2. **Use migrations** for all schema changes
 3. **Use DTOs** when returning data to views or APIs
 4. **Avoid business logic in DbContext**
 5. **Separate configurations** for each entity using `IEntityTypeConfiguration<T>`
 6. **Seed default data** if necessary
-

7.10 Summary of Code-First EF Core

In this module, you learned:

- The **Code-First approach** for EF Core
- How to define models and DbContext
- How to create the database using **migrations**
- How to handle **relationships**: one-to-many and many-to-many
- How to seed initial data
- CRUD operations with DbContext
- Best practices for maintainable, clean code

Code-First EF Core is ideal for **new applications** where you want **full control over models and migrations**, and it integrates perfectly with **ASP.NET Core MVC and APIs**.

Module 8: Session & Cookie Management in ASP.NET Core

Web applications often need to **remember information about users** between requests. HTTP is stateless, meaning every request is independent. To store data between requests, ASP.NET Core provides **sessions** and **cookies**.

This module covers:

- The difference between session and cookies
 - How to configure and use session
 - How to create, read, and delete cookies
 - Best practices for secure storage
-

8.1 Understanding Session vs Cookies

Feature	Session	Cookies
Storage	Server-side	Client-side (browser)
Lifetime	Typically per browser session, configurable	Can persist for days or months
Capacity	Large (limited by server memory)	Small (~4KB per cookie)
Security	Safer, not sent to client	Can be read/modified unless HttpOnly/Secure
Use Case	Temporary user data, shopping cart	Remember login, preferences, tracking

8.2 Configuring Session in ASP.NET Core

Step 1: Install Package (if needed)

Microsoft.AspNetCore.Session

Step 2: Configure in Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDistributedMemoryCache(); // Required for session
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30); // Session expiration
    options.Cookie.HttpOnly = true;                // Prevent JavaScript access
    options.Cookie.IsEssential = true;              // GDPR compliance
});

builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseStaticFiles();
app.UseRouting();
app.UseSession(); // Enable session middleware

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

8.3 Using Session in Controllers

8.3.1 Storing Data

```
HttpContext.Session.SetString("UserName", "John");
HttpContext.Session.SetInt32("CartCount", 5);
```

8.3.2 Reading Data

```
string userName = HttpContext.Session.GetString("UserName");
int? cartCount = HttpContext.Session.GetInt32("CartCount");
```

8.3.3 Removing Data

```
HttpContext.Session.Remove("UserName"); // Remove specific key
```

```
HttpContext.Session.Clear(); // Remove all session data
```

Example: Login Session

```
[HttpPost]
public IActionResult Login(LoginModel model)
{
    if (ModelState.IsValid)
    {
        // Validate user credentials (omitted)
        HttpContext.Session.SetString("UserName", model.Username);
        return RedirectToAction("Dashboard");
    }
    return View(model);
}
```

8.4 Creating and Managing Cookies

Cookies are stored in the **browser** and sent with each request.

8.4.1 Setting Cookies

```
var cookieOptions = new CookieOptions
{
    Expires = DateTime.Now.AddDays(7), // Expiration
    HttpOnly = true,                  // Prevent JavaScript access
    Secure = true                      // HTTPS only
};
```

```
Response.Cookies.Append("AuthToken", "abcdef123456", cookieOptions);
```

8.4.2 Reading Cookies

```
string authToken = Request.Cookies["AuthToken"];
```

8.4.3 Deleting Cookies

```
Response.Cookies.Delete("AuthToken");
```

8.5 Persistent vs Temporary Cookies

- **Persistent Cookies** → Stored on disk; survive browser restarts
- **Temporary Cookies** → Stored in memory; cleared when browser closes

Example:

```
// Persistent cookie
Response.Cookies.Append("Theme", "Dark", new CookieOptions { Expires =
DateTime.Now.AddDays(30) });
```

```
// Temporary cookie
Response.Cookies.Append("SessionToken", "xyz"); // No expiration
```

8.6 Session vs Cookie Example

Imagine a **shopping cart** application:

- Store **cart items** in session (server-side, safer, large data)
- Store **remember me token** in cookie (client-side, persistent login)

```
// Session for temporary cart
HttpContext.Session.SetString("CartItems", cartJson);
```

```
// Cookie for "Remember Me"
Response.Cookies.Append("RememberMe", token, new CookieOptions { Expires =
DateTime.Now.AddDays(14), HttpOnly = true });
```

8.7 Security Considerations

1. **Use HttpOnly cookies** → Prevent JavaScript access, reduce XSS risk
 2. **Use Secure cookies** → Only send over HTTPS
 3. **Encrypt sensitive data** → Do not store passwords in cookies or session
 4. **Set expiration times** → Limit session/cookie lifetime
 5. **Validate all incoming data** → Session or cookie can be tampered
-

8.8 Best Practices

- Prefer **session** for sensitive, temporary data
 - Prefer **cookies** for small, persistent, non-sensitive data
 - Always use **HttpOnly + Secure** flags for authentication tokens
 - Use **Distributed Cache** for session in scalable applications (e.g., Redis)
 - Clean up expired cookies and sessions periodically
-

8.9 Summary of Session & Cookie Management

In this module, you learned:

- The difference between **session and cookies**
- How to configure and use **session in ASP.NET Core**
- How to **store, read, and remove session data**
- How to **create, read, and delete cookies**
- Security considerations and best practices
- Use cases for session vs cookies in real-world applications

By mastering sessions and cookies, you can **track user state**, manage login, shopping carts, and other **stateful features** in your web applications.

Module 9: DTOs and AutoMapper in ASP.NET Core

In modern web applications, it is a **bad practice to expose database entities directly** to the client or views. This can cause security, performance, and maintainability issues. To solve this, developers use **DTOs (Data Transfer Objects)** and **AutoMapper**.

This module covers:

- What DTOs are and why they are important
 - How to create DTOs for MVC and API
 - Using AutoMapper for automatic object mapping
 - Best practices for clean architecture
-

9.1 What Are DTOs (Data Transfer Objects)?

A **DTO** is a simple object that **carries data between layers** of an application:

- From **BLL (Business Logic Layer)** to **API**
- From **API** to **Frontend**
- From **Controller** to **View**

Benefits of Using DTOs

1. **Security** → Do not expose sensitive entity fields
 2. **Performance** → Send only necessary data
 3. **Maintainability** → Decouple API models from database entities
 4. **Consistency** → Standardized data structure for clients
-

9.2 Example: Entity vs DTO

Entity Class (Database Model):

```
public class Student
{
    public int StudentId { get; set; }
    public string Name { get; set; }
```

```
    public string Email { get; set; }
    public string PasswordHash { get; set; } // Sensitive
}
```

DTO Class (For API/Client):

```
public class StudentDTO
{
    public int StudentId { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
}
```

Notice PasswordHash is not included in DTO — this protects sensitive data.

9.3 Creating DTOs in ASP.NET Core

- Create a folder DTOs in your project
- Create separate DTOs for different purposes:

9.3.1 Create DTOs for CRUD Operations

```
public class StudentCreatedDTO
{
    [Required]
    public string Name { get; set; }

    [Required, EmailAddress]
    public string Email { get; set; }

    [Required, MinLength(6)]
    public string Password { get; set; }
}

public class StudentUpdatedDTO
{
    [Required]
    public int StudentId { get; set; }

    [Required]
    public string Name { get; set; }
}
```

9.3.2 Why Separate DTOs?

- **CreatedDTO** → Used when creating a record
- **UpdatedDTO** → Used when updating a record

- **ReadDTO** → Used when returning data to API/Views

This separation ensures **validation is specific to operation**.

9.4 Introduction to AutoMapper

AutoMapper is a library that **automatically maps properties** between objects.

- Reduces repetitive code like:

```
var dto = new StudentDTO
{
    StudentId = student.StudentId,
    Name = student.Name,
    Email = student.Email
};
```

- AutoMapper does this automatically with configuration.
-

9.5 Installing AutoMapper

```
AutoMapper
AutoMapper.Extensions.Microsoft.DependencyInjection
```

9.6 Configuring AutoMapper in ASP.NET Core

Step 1: Create a Mapping Profile

```
using AutoMapper;

public class MappingProfile : Profile
{
    public MappingProfile()
    {
        CreateMap<Student, StudentDTO>();
        CreateMap<StudentCreateDTO, Student>();
        CreateMap<StudentUpdateDTO, Student>();
    }
}
```

Step 2: Register AutoMapper

In Program.cs:

```
builder.Services.AddAutoMapper(typeof(MappingProfile));
```

9.7 Using AutoMapper in Controllers

9.7.1 Inject IMapper

```
private readonly IMapper _mapper;

public StudentsController(SchoolDbContext context, IMapper mapper)
{
    _context = context;
    _mapper = mapper;
}
```

9.7.2 Mapping Entities to DTOs

```
public IActionResult GetStudents()
{
    var students = _context.Students.ToList();
    var studentDTOs = _mapper.Map<List<StudentDTO>>(students);
    return Ok(studentDTOs); // Return JSON to API
}
```

9.7.3 Mapping DTO to Entity

```
[HttpPost]
public IActionResult Create(StudentCreateDTO dto)
{
    if (!ModelState.IsValid) return BadRequest(ModelState);

    var student = _mapper.Map<Student>(dto);
    _context.Students.Add(student);
    _context.SaveChanges();

    var resultDTO = _mapper.Map<StudentDTO>(student);
    return Ok(resultDTO);
}
```

9.8 Advanced AutoMapper Features

- **Flattening** → Map nested objects automatically
- **Custom Mapping** → Use `.ForMember()` for complex rules
- **Reverse Mapping** → Map DTO back to Entity easily

Example: Custom Mapping

```
CreateMap<Student, StudentDTO>()  
    .ForMember(dest => dest.Name, opt => opt.MapFrom(src => src.Name.ToUpper()));
```

9.9 Best Practices for DTOs & AutoMapper

1. **Never expose EF entities directly**
 2. **Create separate DTOs for different operations**
 3. **Use AutoMapper to reduce repetitive mapping code**
 4. **Keep DTOs lightweight** (only necessary properties)
 5. **Validate DTOs** using Data Annotations before mapping
-

9.10 Summary of DTOs and AutoMapper

In this module, you learned:

- What **DTOs** are and why they are essential for secure, maintainable applications
- How to create **Create/Update/Read DTOs**
- How **AutoMapper** simplifies mapping between entities and DTOs
- How to configure AutoMapper using **Profiles**
- How to integrate AutoMapper in controllers for **CRUD operations**
- Best practices to keep your architecture **clean and secure**

Using DTOs and AutoMapper ensures your **API or MVC layer is decoupled from database models**, making your application more maintainable and secure.

Module 10: Authentication with Custom Authorization in ASP.NET Core MVC

Securing a web application is crucial. In many ASP.NET Core MVC applications, you need **persistent user login** and **role-based or custom authorization**. One popular approach is **cookie-based authentication** combined with **session management**. Additionally, you may want to implement **custom authorization** using a **filter attribute**, so you can apply it directly to controllers or actions.

This module covers:

- Session- and cookie-based authentication
 - Maintaining user login state
 - Creating a custom authorization filter
 - Applying authorization via annotations
 - Best practices for secure implementation
-

10.1 Understanding Session and Cookie-Based Authentication

Feature	Session	Cookie
Storage	Server-side	Client-side (browser)
Lifetime	Temporary (per session)	Persistent (custom expiration)
Security	Safer, not exposed to client	Can be encrypted and HttpOnly
Use Case	Temporary user state	Remember login, user roles, auth tokens

Goal: Store user identity in **session** and **cookies**, then **check authorization** for each request via a filter attribute.

10.2 Setting Up Session and Cookie Authentication

Step 1: Configure Services in Program.cs

```
builder.Services.AddDistributedMemoryCache();

builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});

builder.Services.AddAuthentication("MyCookieAuth")
    .AddCookie("MyCookieAuth", options =>
    {
        options.Cookie.Name = "UserLoginCookie";
        options.LoginPath = "/Account/Login";
        options.AccessDeniedPath = "/Account/AccessDenied";
        options.ExpireTimeSpan = TimeSpan.FromMinutes(30);
    });

builder.Services.AddControllersWithViews();
```

Step 2: Enable Middleware

```
app.UseSession();
app.UseAuthentication();
app.UseAuthorization();
```

10.3 User Login: Session + Cookie

Step 1: Login Action

```
[HttpPost]
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (ModelState.IsValid)
    {
        // Validate user credentials (database or in-memory)
        var user = AuthenticateUser(model.Username, model.Password);
        if (user != null)
        {
            // Create claims
            var claims = new List<Claim>
            {
                new Claim(ClaimTypes.Name, user.Username),
                new Claim(ClaimTypes.Role, user.Role)
            };
        }
    }
}
```

```

        var identity = new ClaimsIdentity(claims, "MyCookieAuth");
        var principal = new ClaimsPrincipal(identity);

        // Sign in using cookie
        await HttpContext.SignInAsync("MyCookieAuth", principal);

        // Store some user info in session
        HttpContext.Session.SetString("UserRole", user.Role);

        return RedirectToAction("Dashboard", "Home");
    }

    ModelState.AddModelError("", "Invalid credentials");
}
return View(model);
}

```

Step 2: Logout Action

```

public async Task<IActionResult> Logout()
{
    await HttpContext.SignOutAsync("MyCookieAuth");
    HttpContext.Session.Clear();
    return RedirectToAction("Login", "Account");
}

```

This ensures both **session and cookie are cleared** on logout.

10.4 Custom Authorization Attribute

Sometimes you want to **restrict access to certain actions** based on session data or roles. You can create a **custom filter attribute**.

Step 1: Create the Filter Attribute

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

public class CustomAuthorizeAttribute : Attribute, IAuthorizationFilter
{
    private readonly string _role;

    public CustomAuthorizeAttribute(string role = "")
    {
        _role = role;
    }
}

```

```

public void OnAuthorization(AuthorizationFilterContext context)
{
    // Check if user is authenticated
    var isAuthenticated = context.HttpContext.User.Identity.IsAuthenticated;
    if (!isAuthenticated)
    {
        context.Result = new RedirectToActionResult("Login", "Account", null);
        return;
    }

    // Check session for role if specified
    if (!string.IsNullOrEmpty(_role))
    {
        var sessionRole = context.HttpContext.Session.GetString("UserRole");
        if (sessionRole != _role)
        {
            context.Result = new RedirectToActionResult("AccessDenied", "Account",
null);
            return;
        }
    }
}

```

Step 2: Apply the Attribute

You can apply it to **controllers** or **actions**:

```

[CustomAuthorize("Admin")]
public IActionResult AdminPanel()
{
    return View();
}

[CustomAuthorize] // Any authenticated user
public IActionResult Dashboard()
{
    return View();
}

```

The attribute checks both **authentication via cookie** and **authorization via session role**.

10.5 Access Denied Page

Create a simple **AccessDenied** view for unauthorized users:

```

public IActionResult AccessDenied()

```

```
{  
    return View(); // Inform user they do not have permission  
}
```

10.6 Advantages of This Approach

1. **No need to manually check authorization in every action** → centralized in filter
 2. **Session + cookie** ensures user identity and role persist across requests
 3. Works for **MVC web apps**, easy to integrate with dashboard, admin panels
 4. Customizable to **any role or condition**
-

10.7 Best Practices

- Always set **HttpOnly** and **Secure** cookies
 - Keep **session size small** (only store essential info like role or userId)
 - Centralize authorization logic in **custom filter attributes**
 - Clear session and cookie on logout
 - Use **claims for role info** as backup to session
-

10.8 Summary of Module 10

In this module, you learned:

- How to implement **session and cookie-based authentication** in MVC Core
- How to maintain user login state using **session and cookie**
- How to create a **custom authorization attribute** to protect actions and controllers
- How to apply the attribute as an **annotation** for declarative authorization
- Best practices to keep authentication and authorization **secure and maintainable**

This approach allows you to **control access** in a flexible and reusable way, combining **server-side session data** with **cookie-based identity** for secure web applications.